Welcome to

Become a Python Developer

Course Instructor: Md. Azizul Hakim

Python List

lists are used to store multiple data at once

A list can:

- **Store elements of different types**
- **Store duplicate elements**

```
#list example
ages =  [19, 26, 23]

print(ages)


# list with elements of different data types
list1 = [1, "Hello", 3.4]

# list with duplicate elements
list1 = [1, "Hello", 3.4, "Hello", 1]

# empty list
list3 = []
```

List Index

lists are ordered and each item in a list is associated with a number known as a list index.

The index of the first element is 0

```
languages = ["Python", "Swift", "C++"]

# access item at index 0
print(languages[0])   # Python

# access item at index 2
print(languages[2])   # C++
```

```
#negative index
languages = ["Python", "Swift", "C++"]

# access item at index 0
print(languages[-1])   # C++

# access item at index 2
print(languages[-3])   # Python
```

```
# List slicing in Python

my_list = ['p','r','o','g','r','a','m','i','z']

# items from index 2 to index 4
print(my_list[2:5])

# items from index 5 to end
print(my_list[5:])

# items beginning to end
print(my_list[:])
```

List Insert(add)

append()

```
numbers = [21, 34, 54, 12]

print("Before Append:", numbers)

# using append method
numbers.append(32)

print("After Append:", numbers)
```

extend()

```
numbers = [1, 3, 5]

even_numbers = [4, 6, 8]

# add elements of even_numbers to the numbers list
numbers.extend(even_numbers)

print("List after append:", numbers)
```

Insert()

```
numbers = [10, 30, 40]

# insert an element at index 1 (second position)
numbers.insert(1, 20)

print(numbers) # [10, 20, 30, 40]
```

List Delete(remove)

Using del statement

```
languages = ['Python', 'Swift', 'C++', 'C', 'Java', 'Rust', 'R']

# deleting the second item
del languages[1]
print(languages) # ['Python', 'C++', 'C', 'Java', 'Rust', 'R']

# deleting the last item
del languages[-1]
print(languages) # ['Python', 'C++', 'C', 'Java', 'Rust']

# delete the first two items
del languages[0 : 2]  # ['C', 'Java', 'Rust']
print(languages)
```

Using remove

```
languages = ['Python', 'Swift', 'C++',
'C', 'Java', 'Rust', 'R']

# remove 'Python' from the list
languages.remove('Python')

print(languages) # ['Swift', 'C++', 'C',
'Java', 'Rust', 'R']
```

List- more methods

Check existence

```
languages = ['Python', 'Swift', 'C++']

print('C' in languages)    # False
print('Python' in languages)    # True
```

List length

```
languages = ['Python', 'Swift', 'C++']

print("List: ", languages)

print("Total Elements: ",
len(languages))    # 3
```

List comprehension

```
# create a list with value n ** 2 where n is a number from 1 to 5
numbers = [n**2 for n in range(1, 6)]

print(numbers)

# Output: [1, 4, 9, 16, 25]
```

Python Tuple

Same as list

Only difference is immutable

```
# Different types of tuples

# Empty tuple
my_tuple = ()
print(my_tuple)

# Tuple having integers
my_tuple = (1, 2, 3)
print(my_tuple)

# tuple with mixed datatypes
my_tuple = (1, "Hello", 3.4)
print(my_tuple)

# nested tuple
my_tuple = ("mouse", [8, 4, 6], (1, 2, 3))
print(my_tuple)
```

Tuple or List

- tuples for heterogeneous (different) data types and lists for homogeneous (similar) data types.
- Since tuples are immutable, iterating through a tuple is faster than with a list. So there is a slight performance boost.
- Tuples that contain immutable elements can be used as a key for a dictionary. With lists, this is not possible.
- If you have data that doesn't change, implementing it as tuple will guarantee that it remains write-protected.

Python Dictionary

- a dictionary is a collection that allows us to store data in **key-value** pairs.
- A dictionary is a collection which is ordered*, changeable and do not allow duplicates.

```
# dictionary

thisdict =      {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict["brand"])

#duplicate value override

thisdict ={
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964,
  "year": 2020
}
print(thisdict)
```

Python Strings

a string is a sequence of characters

single quotes or double quotes is used to represent a string in Python

strings are immutable means the characters of a string cannot be changed

```
# create a string using double quotes
string1 = "Python programming"

# create a string using single quotes
string1 = 'Python programming'
```

```
message = 'Hola Amigos'
message[0] = 'H'
print(message)
```

Access string character

Same as list indexing

```
greet = 'hello'

# access 1st index element
print(greet[1]) # "e“

#negative indexing
greet = 'hello'

# access 4th last element
print(greet[-4]) # "e"
```

slicing

```
greet = 'Hello'

# access character from 1st index to
3rd index
print(greet[1:4])  # "ell"
```

String Operations

Compare two string

```
str1 = “ai Quest"
str2 = "Python."
str3 = “ai Quest"

# compare str1 and str2
print(str1 == str2)

# compare str1 and str3
print(str1 == str3)
```

Join strings

```
greet = "Hello, "
name = “Sanam"

# using + operator
result = greet + name
print(result)

# Output: Hello, Sanam
```

Strings (more)

Escape Sequence

```
# escape double quotes
example = "He said, \"What's there?\""

# escape single quotes
example = 'He said, "What\'s there?"'

print(example)

# Output: He said, "What's there?"
```

f strings

```
sub = ‘machine learning'
lan = ‘Python'

print(f'{sub} is from {lan}')
```

String length

```
name = ‘Sanam'

# count length of greet string
print(len(name))

# Output: 5
```

String methods

there are various string methods present in Python

- **upper()**
- **lower()**
- **partition()**
- **replace()**
- **find()**
- **rstrip()**
- **split()**
- **startswith()**
- **isnumeric()**
- **index()**